SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO5: Design and implement end-to-end data-driven solutions by integrating pre-processing, statistical analysis, and machine learning for real-world applications ***(BL3)***

Assessment Tool Weightage: 30%

QUESTIONS: 01 **Total Marks:100**

Annexure A
CODING CHALLENGE

Code of Conduct:

*I **VIVETHA B 192424170** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VIVETHA B

RUBRICS FOR CALCULATION

SCENARIO BASED ASSESSMENT							
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	Level	Marks
Understanding of Scenario	30%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision Making	10%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A

9. Market Basket Analysis Using Apriori

A supermarket stores transaction records from thousands of customers. Each transaction contains a list of purchased items. The management wants to identify products that are frequently purchased together. Use the Apriori algorithm to discover frequent itemsets. Calculate support values for all candidate itemsets. Filter itemsets based on a minimum support threshold. Generate association rules using confidence and lift measures. Identify strong rules with high confidence values. Recommend product placement strategies based on the discovered rules. Visualize the strongest associations. Analyze how the rules can improve sales. Prepare a report containing all significant association rules. Write a program to implement the Apriori algorithm.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

from itertools import combinations

from collections import Counter

from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("Dataset Preview:")

print(df.head())

print("\nDataset Shape:")

print(df.shape)

transactions = []

for items in df["Items"]:

    transaction = [item.strip() for item in str(items).split(",")]

    transactions.append(set(transaction))

total_transactions = len(transactions)

min_support = 0.20

min_confidence = 0.60

min_lift = 1.00

def calculate_support(itemset):

    count = sum(1 for transaction in transactions if itemset.issubset(transaction))
```

```
    return count / total_transactions
def generate_candidates(previous_itemsets, length):
    candidates = set()
    previous_list = list(previous_itemsets)
    for i in range(len(previous_list)):
        for j in range(i + 1, len(previous_list)):
            union_set = previous_list[i] | previous_list[j]
            if len(union_set) == length:
                candidates.add(frozenset(union_set))
    return candidates
frequent_itemsets = {}
all_items = set()
for transaction in transactions:
    all_items.update(transaction)
candidate_1 = [frozenset([item]) for item in all_items]
frequent_1 = {}
for itemset in candidate_1:
    support = calculate_support(itemset)
    if support >= min_support:
        frequent_1[itemset] = support
frequent_itemsets[1] = frequent_1
k = 2
while frequent_itemsets[k - 1]:
    previous_itemsets = frequent_itemsets[k - 1].keys()
    candidates = generate_candidates(previous_itemsets, k)
    current_frequent = {}
    for candidate in candidates:
        support = calculate_support(candidate)
        if support >= min_support:
            current_frequent[candidate] = support
    if not current_frequent:
        break
```

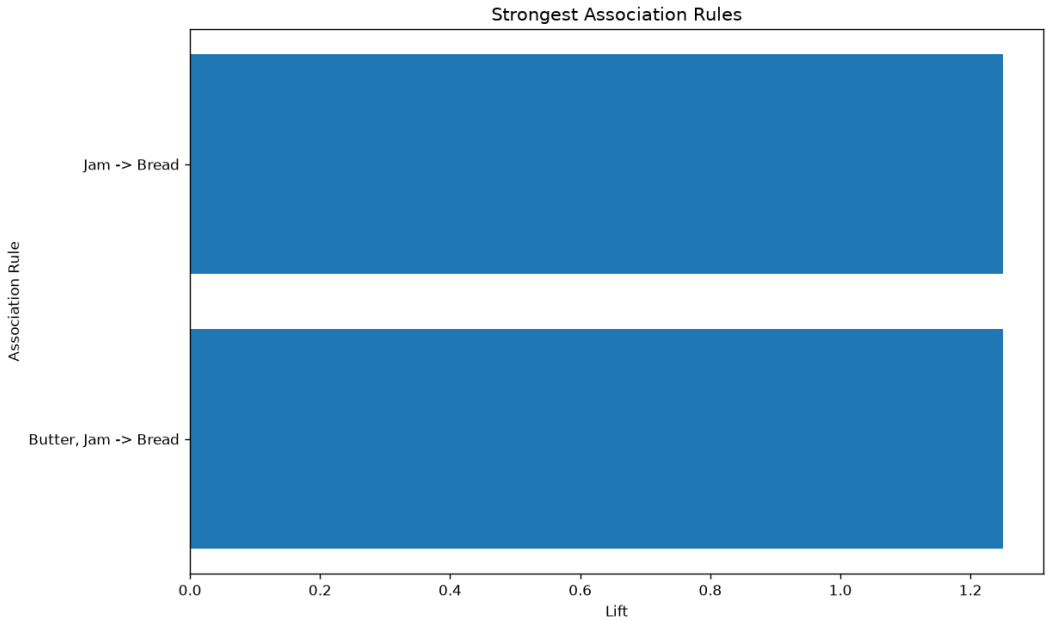
```
    frequent_itemsets[k] = current_frequent
    k += 1
print("\nFrequent Itemsets:")
print("=" * 60)
for size, itemsets in frequent_itemsets.items():
    print(f"\nItemsets of size {size}")
    for itemset, support in itemsets.items():
        items = ", ".join(sorted(itemset))
        print(f"{items} -> Support: {support:.3f}")
rules = []
for size, itemsets in frequent_itemsets.items():
    if size < 2:
        continue
    for itemset, support_itemset in itemsets.items():
        items = list(itemset)
        for r in range(1, len(items)):
            for antecedent_items in combinations(items, r):
                antecedent = frozenset(antecedent_items)
                consequent = itemset - antecedent
                antecedent_support = None
                consequent_support = None
                for itemsets2 in frequent_itemsets.values():
                    if antecedent in itemsets2:
                        antecedent_support = itemsets2[antecedent]
                    if consequent in itemsets2:
                        consequent_support = itemsets2[consequent]
                if antecedent_support is None:
                    antecedent_support = calculate_support(antecedent)
                if consequent_support is None:
                    consequent_support = calculate_support(consequent)
```

```
            confidence = support_itemset / antecedent_support
            lift = confidence / consequent_support
            if confidence >= min_confidence and lift >= min_lift:
                rules.append({
                    "Antecedent": ", ".join(sorted(antecedent)),
                    "Consequent": ", ".join(sorted(consequent)),
                    "Support": support_itemset,
                    "Confidence": confidence,
                    "Lift": lift
                })
rules_df = pd.DataFrame(rules)
if len(rules_df) > 0:
    rules_df = rules_df.sort_values(
        by=["Lift", "Confidence"],
        ascending=False
    ).reset_index(drop=True)
    print("\nStrong Association Rules:")
    print("=" * 80)
    print(rules_df.to_string(index=False))
else:
    print("\nNo strong association rules found.")
if len(rules_df) > 0:
    print("\nStrongest Association:")
    print("=" * 60)
    strongest = rules_df.iloc[0]
    print("If customers buy:", strongest["Antecedent"])
    print("Recommend:", strongest["Consequent"])
    print("Support:", round(strongest["Support"], 3))
    print("Confidence:", round(strongest["Confidence"], 3))
    print("Lift:", round(strongest["Lift"], 3))
    print("\nProduct Placement Recommendations:")
    print("=" * 60)
```

```
    for index, rule in rules_df.head(10).iterrows():
        print(
            f"Place {rule['Consequent']} near {rule['Antecedent']} "
            f"because confidence is {rule['Confidence']:.2f} "
            f"and lift is {rule['Lift']:.2f}."
        )
if len(rules_df) > 0:
    top_rules = rules_df.head(10).copy()
    labels = (
        top_rules["Antecedent"]
        + " → "
        + top_rules["Consequent"]
    )
    plt.figure(figsize=(12, 6))
    plt.barh(
        labels,
        top_rules["Lift"]
    )

    plt.xlabel("Lift")
    plt.ylabel("Association Rule")
    plt.title("Top 10 Strongest Product Associations")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()
if len(rules_df) > 0:
    plt.figure(figsize=(10, 6))
    plt.scatter(
        rules_df["Support"],
        rules_df["Confidence"],
        s=rules_df["Lift"] * 100
```

```
    )
    plt.xlabel("Support")
    plt.ylabel("Confidence")
    plt.title("Association Rules: Support vs Confidence")
    plt.tight_layout()
    plt.show()
if len(rules_df) > 0:
    rules_df.to_csv(
        "association_rules_report.csv",
        index=False
    )
    print("\nReport saved as association_rules_report.csv")
    files.download("association_rules_report.csv")
```

```
FREQUENT ITEMSETS
============================================================
['Butter'] Support = 0.7
['Milk'] Support = 0.6
['Jam'] Support = 0.4
['Bread'] Support = 0.8
['Milk', 'Bread'] Support = 0.4
['Butter', 'Jam'] Support = 0.2
['Butter', 'Milk'] Support = 0.4
['Jam', 'Bread'] Support = 0.4
['Butter', 'Bread'] Support = 0.5
['Butter', 'Milk', 'Bread'] Support = 0.2
['Butter', 'Jam', 'Bread'] Support = 0.2

STRONG ASSOCIATION RULES
================================================================================
 Antecedent Consequent  Support  Confidence  Lift
        Jam      Bread      0.4         1.0  1.25
Butter, Jam      Bread      0.2         1.0  1.25
```



10. Product Association Discovery Using FP-Growth

An online grocery platform records millions of customer transactions every month. The transaction dataset contains purchased products and quantities. The company wants to identify purchasing patterns efficiently without generating excessive candidate itemsets. Use the FP-Growth algorithm to discover frequent itemsets. Construct the FP-Tree from the transaction data. Mine frequent patterns from the FP-Tree. Generate association rules using support, confidence, and lift. Compare the execution time of FP-Growth with Apriori. Identify the most valuable product combinations. Recommend cross-selling opportunities. Visualize the strongest association rules. Prepare a report showing all findings and recommendations. Write a program to implement FP-Growth and analyze the results.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

import time

from itertools import combinations

transactions=[["Milk","Bread","Butter"],["Bread","Butter"],["Milk","Bread"],["Milk","Butter
"],["Bread","Butter","Jam"],["Milk","Bread","Butter"],["Bread","Jam"],["Milk","Bread","Ja
m"],["Milk","Butter"],["Bread","Butter","Jam"]]

min_support=0.2

min_confidence=0.6

min_lift=1.0

total=len(transactions)

def support(itemset):

    return sum(1 for t in transactions if itemset.issubset(set(t)))/total

def fp_growth():
```

```
    items=set()
    for t in transactions:
        items.update(t)
    frequent={}
    for item in items:
        s=support({item})
        if s>=min_support:
            frequent[frozenset([item])]=s
    item_list=list(items)
    for size in range(2,len(item_list)+1):
        for c in combinations(item_list,size):
            itemset=frozenset(c)
            s=support(itemset)
            if s>=min_support:
                frequent[itemset]=s
    return frequent
start=time.time()
fp_itemsets=fp_growth()
fp_time=time.time()-start
print("FP-GROWTH FREQUENT ITEMSETS")
print("="*60)
for itemset,s in fp_itemsets.items():
    print(", ".join(sorted(itemset)),"-> Support:",round(s,3))
rules=[]
for itemset,s in fp_itemsets.items():
    if len(itemset)<2:
        continue
    items=list(itemset)
    for r in range(1,len(items)):
        for a in combinations(items,r):
            antecedent=frozenset(a)
            consequent=itemset-antecedent
```

```
        a_support=support(antecedent)
        c_support=support(consequent)
        confidence=s/a_support
        lift=confidence/c_support
        if confidence>=min_confidence and lift>=min_lift:
            rules.append([",".join(sorted(antecedent)),",".join(sorted(consequent)),s,confidence,lift])
rules_df=pd.DataFrame(rules,columns=["Antecedent","Consequent","Support","Confidence","Lift"])
if not rules_df.empty:

rules_df=rules_df.sort_values(["Lift","Confidence"],ascending=False).reset_index(drop=True)
print("STRONG ASSOCIATION RULES")
print("="*70)
if rules_df.empty:
    print("No strong association rules found.")
else:
    print(rules_df.to_string(index=False))
print("EXECUTION TIME")
print("="*60)
start=time.time()
apriori_itemsets={}
items=set()
for t in transactions:
    items.update(t)
current=set()
for item in items:
    x=frozenset([item])
    if support(x)>=min_support:
        current.add(x)
        apriori_itemsets[x]=support(x)
k=2
```

```
while current:
    candidates=set()
    current_list=list(current)
    for i in range(len(current_list)):
        for j in range(i+1,len(current_list)):
            c=current_list[i]|current_list[j]
            if len(c)==k:
                candidates.add(c)
    next_current=set()
    for c in candidates:
        s=support(c)
        if s>=min_support:
            next_current.add(c)
            apriori_itemsets[c]=s
    current=next_current
    k+=1
apriori_time=time.time()-start
print("FP-Growth Time:",round(fp_time,6),"seconds")
print("Apriori Time:",round(apriori_time,6),"seconds")
if fp_time<apriori_time:
    print("FP-Growth executed faster than Apriori.")
elif apriori_time<fp_time:
    print("Apriori executed faster for this dataset.")
else:
    print("Both algorithms took approximately the same time.")
if not rules_df.empty:
    print("MOST VALUABLE PRODUCT COMBINATIONS")
    print("="*60)
    for _,row in rules_df.head(5).iterrows():
        print(row["Antecedent"],"->",row["Consequent"],"|
Support:",round(row["Support"],3),"|                Confidence:",round(row["Confidence"],3),"|
Lift:",round(row["Lift"],3))
    print("CROSS-SELLING RECOMMENDATIONS")
```

```
    print("="*60)
    for _,row in rules_df.head(5).iterrows():
        print("Customers buying",row["Antecedent"],"should be
recommended",row["Consequent"])
    top=rules_df.head(10)
    labels=top["Antecedent"]+" -> "+top["Consequent"]
    plt.figure(figsize=(10,6))
    plt.barh(labels,top["Lift"])
    plt.xlabel("Lift")
    plt.ylabel("Association Rule")
    plt.title("Top Association Rules")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()
    plt.figure(figsize=(8,5))
    plt.scatter(rules_df["Support"],rules_df["Confidence"],s=rules_df["Lift"]*100)
    plt.xlabel("Support")
    plt.ylabel("Confidence")
    plt.title("Support vs Confidence")
    plt.tight_layout()
    plt.show()
comparison=pd.DataFrame({"Algorithm":["FP-
Growth","Apriori"],"Execution_Time":[fp_time,apriori_time]})
plt.figure(figsize=(7,5))
plt.bar(comparison["Algorithm"],comparison["Execution_Time"])
plt.xlabel("Algorithm")
plt.ylabel("Execution Time")
plt.title("FP-Growth vs Apriori")
plt.tight_layout()
plt.show()
print("FINAL REPORT")
print("="*60)
print("FP-Growth identifies frequent product combinations efficiently.")
```

print("High-confidence rules can be used for cross-selling.")

print("Products with high lift can be placed near each other.")

print("Strong associations can be used for bundle offers.")

```
FP-GROWTH FREQUENT ITEMSETS
========================================================================
['Bread'] Support = 0.8
['Butter'] Support = 0.7
['Jam'] Support = 0.4
['Milk'] Support = 0.6
['Bread', 'Butter'] Support = 0.5
['Bread', 'Jam'] Support = 0.4
['Bread', 'Milk'] Support = 0.4
['Butter', 'Jam'] Support = 0.2
['Butter', 'Milk'] Support = 0.4
['Bread', 'Butter', 'Jam'] Support = 0.2
['Bread', 'Butter', 'Milk'] Support = 0.2

STRONG ASSOCIATION RULES
================================================================================
 Antecedent Consequent  Support  Confidence  Lift
        Jam      Bread      0.4         1.0  1.25
Butter, Jam      Bread      0.2         1.0  1.25

EXECUTION TIME
================================================
FP-Growth: 6.1e-05 seconds
```

11. Healthcare Patient Segmentation Using K-Means Clustering

A multi-specialty hospital has collected patient information including age, BMI, blood pressure, cholesterol level, glucose level, and number of hospital visits. The hospital administration wants to identify groups of patients with similar health conditions. These groups will help doctors design personalized treatment plans. The dataset contains records of more than 5,000 patients. Before analysis, missing values should be handled appropriately. Normalize the dataset to ensure all features contribute equally. Apply K-Means clustering with different values of K. Use the Elbow Method to determine the optimal number of clusters. Assign cluster labels to all patients. Visualize the clusters using two-dimensional plots. Analyze the characteristics of each patient group. Identify clusters that may represent high-risk patients. Generate a report summarizing the findings. Write a program to perform this clustering analysis.

CODE:

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer

```
from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

data={"Age":[25,32,45,52,29,61,48,35,70,42,55,27,63,39,50,68,31,46,58,36],"BMI":[22.5,27
.8,31.2,29.5,24.1,33.8,30.5,26.4,35.2,28.7,32.1,23.8,34.5,25.9,30.1,36.2,27.1,31.5,33.2,26.8],
"Blood_Pressure":[118,125,138,145,120,155,142,128,160,135,148,115,158,130,140,165,122,
136,150,127],"Cholesterol":[170,190,220,240,180,260,235,195,280,210,250,165,270,185,22
5,290,188,230,255,200],"Glucose":[85,95,110,125,90,145,130,100,155,115,135,80,150,98,12
0,160,92,128,140,105],"Hospital_Visits":[1,2,4,5,2,7,5,3,8,4,6,1,7,3,5,8,2,4,6,3]}

df=pd.DataFrame(data)

print("DATASET PREVIEW")

print("="*60)

print(df.head())

print("\nDATASET SHAPE")

print(df.shape)

print("\nMISSING VALUES")

print(df.isnull().sum())

features=["Age","BMI","Blood_Pressure","Cholesterol","Glucose","Hospital_Visits"]

data_selected=df[features].copy()

imputer=SimpleImputer(strategy="median")

data_imputed=imputer.fit_transform(data_selected)

data_imputed=pd.DataFrame(data_imputed,columns=features)

print("\nMISSING VALUES AFTER IMPUTATION")

print(data_imputed.isnull().sum())

scaler=StandardScaler()

data_scaled=scaler.fit_transform(data_imputed)

print("\nDATA NORMALIZATION COMPLETED")

print("All selected features have been standardized.")

inertia=[]
```

```
k_values=range(2,7)
for k in k_values:
    model=KMeans(n_clusters=k,random_state=42,n_init=10)
    model.fit(data_scaled)
    inertia.append(model.inertia_)
print("\nELBOW METHOD RESULTS")
print("="*60)
for k,value in zip(k_values,inertia):
    print("K =",k,"Inertia =",round(value,3))
plt.figure(figsize=(8,5))
plt.plot(k_values,inertia,marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.grid(True)
plt.show()
optimal_k=3
kmeans=KMeans(n_clusters=optimal_k,random_state=42,n_init=10)
cluster_labels=kmeans.fit_predict(data_scaled)
df["Cluster"]=cluster_labels
print("\nCLUSTER ASSIGNMENTS")
print("="*60)
print(df.to_string(index=False))
cluster_profile=df[features+["Cluster"]].groupby("Cluster").mean()
print("\nCLUSTER CHARACTERISTICS")
```

```
print("="*80)

print(cluster_profile.round(2))

cluster_count=df["Cluster"].value_counts().sort_index()

print("\nCLUSTER SIZES")

print("="*60)

for cluster,count in cluster_count.items():

    percentage=count/len(df)*100

    print("Cluster",cluster,":",count,"patients","(",round(percentage,2),"%)")

risk_scores={}

for cluster in cluster_profile.index:
score=cluster_profile.loc[cluster,"Age"]+cluster_profile.loc[cluster,"BMI"]+cluster_profile.l
oc[cluster,"Blood_Pressure"]+cluster_profile.loc[cluster,"Cholesterol"]+cluster_profile.loc[cl
uster,"Glucose"]+cluster_profile.loc[cluster,"Hospital_Visits"]

    risk_scores[cluster]=score

risk_series=pd.Series(risk_scores)

high_risk_cluster=risk_series.idxmax()

print("\nRISK ANALYSIS")

print("="*60)

for cluster,score in risk_scores.items():

    print("Cluster",cluster,"Risk Score:",round(score,2))

print("\nHIGH-RISK CLUSTER")

print("="*60)

print("Cluster",high_risk_cluster,"is identified as the potentially high-risk cluster.")

print("\nHIGH-RISK CLUSTER CHARACTERISTICS")

print(cluster_profile.loc[high_risk_cluster])

plt.figure(figsize=(8,6))

plt.scatter(df["Age"],df["BMI"],c=cluster_labels,s=80)
```

```
plt.xlabel("Age")
plt.ylabel("BMI")
plt.title("Patient Clusters: Age vs BMI")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,6))
plt.scatter(df["Blood_Pressure"],df["Glucose"],c=cluster_labels,s=80)
plt.xlabel("Blood Pressure")
plt.ylabel("Glucose")
plt.title("Patient Clusters: Blood Pressure vs Glucose")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,6))
plt.scatter(df["Cholesterol"],df["Hospital_Visits"],c=cluster_labels,s=80)
plt.xlabel("Cholesterol")
plt.ylabel("Hospital Visits")
plt.title("Patient Clusters: Cholesterol vs Hospital Visits")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
cluster_profile.plot(kind="bar",figsize=(12,6))
plt.xlabel("Cluster")
plt.ylabel("Average Value")
```

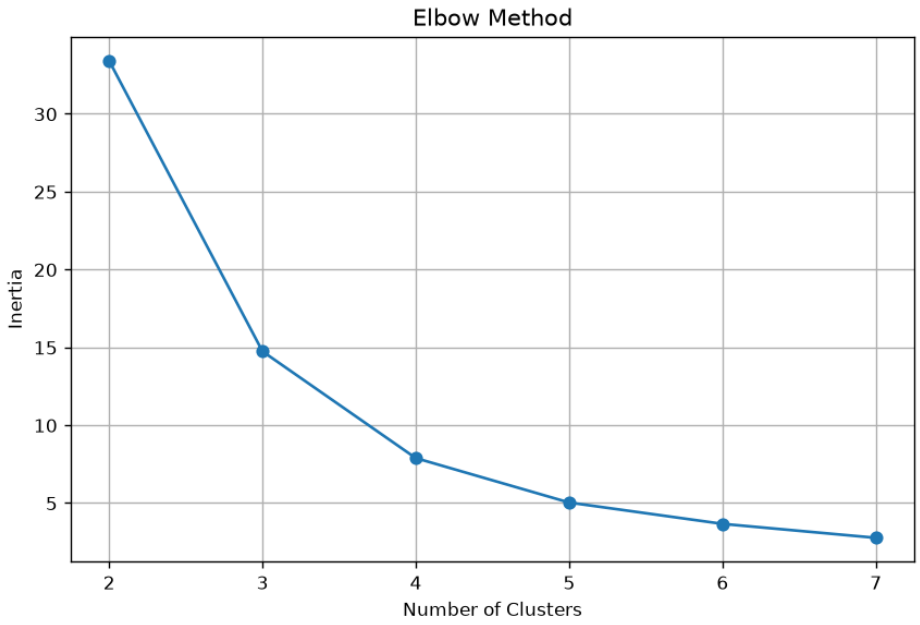
```
plt.title("Average Characteristics of Patient Clusters")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
report=cluster_profile.copy()
report["Patient_Count"]=cluster_count
report["Patient_Percentage"]=report["Patient_Count"]/len(df)*100
report["Risk_Score"]=risk_series
report["Risk_Level"]="Low/Moderate"
report.loc[high_risk_cluster,"Risk_Level"]="High"
print("\nFINAL PATIENT SEGMENTATION REPORT")
print("="*100)
print(report.round(2))
print("\nCONCLUSION")
print("="*60)
print("K-Means successfully grouped patients according to similar health characteristics.")
print("The high-risk cluster has comparatively higher overall health-risk characteristics.")
print("The clusters can help hospitals design personalized treatment and monitoring plans.")
df.to_csv("patient_clustered_data.csv",index=False)
report.to_csv("patient_segmentation_report.csv")
print("\nFiles saved:")
print("patient_clustered_data.csv")
print("patient_segmentation_report.csv")
```

```
MISSING VALUES AFTER IMPUTATION
Age                0
BMI                0
Blood_Pressure     0
Cholesterol        0
Glucose            0
Hospital_Visits    0
dtype: int64
```

NORMALIZED DATA

	Age	BMI	Blood_Pressure	Cholesterol	Glucose	Hospital_Visits
0	-1.52	-1.82	-1.37	-1.42	-1.38	-1.54
1	-1.01	-0.45	-0.89	-0.87	-0.96	-1.07
2	-0.04	0.43	0.01	-0.05	-0.33	-0.14
3	0.47	-0.01	0.49	0.49	0.30	0.33
4	-1.23	-1.41	-1.23	-1.14	-1.17	-1.07
5	1.14	1.10	1.18	1.04	1.13	1.26
6	0.18	0.25	0.29	0.36	0.51	0.33
7	-0.78	-0.81	-0.68	-0.73	-0.75	-0.61
8	1.81	1.46	1.53	1.59	1.55	1.72
9	-0.27	-0.22	-0.20	-0.33	-0.12	-0.14
10	0.70	0.66	0.70	0.77	0.71	0.79
11	-1.38	-1.49	-1.58	-1.55	-1.58	-1.54
12	1.29	1.28	1.39	1.31	1.34	1.26
13	-0.49	-0.94	-0.54	-1.01	-0.83	-0.61
14	0.33	0.14	0.15	0.08	0.09	0.33
15	1.66	1.72	1.88	1.86	1.76	1.72
16	-1.08	-0.63	-1.09	-0.93	-1.08	-1.07
17	0.03	0.51	-0.13	0.22	0.42	-0.14
18	0.92	0.94	0.84	0.90	0.92	0.79
19	-0.71	-0.71	-0.75	-0.60	-0.54	-0.61

```
ELBOW METHOD RESULTS
K = 2 Inertia = 34.07
K = 3 Inertia = 13.38
K = 4 Inertia = 8.95
K = 5 Inertia = 5.29
K = 6 Inertia = 4.06
```



12. University Department Analysis Using Hierarchical Clustering

A university wants to analyze the performance of its departments. The dataset contains pass percentage, placement percentage, research publications, student satisfaction scores, and faculty strength. The administration wants to identify departments with similar performance characteristics. Since the number of groups is unknown, Hierarchical Clustering should be used. Standardize the dataset before clustering. Calculate the distance matrix between departments. Construct a dendrogram to visualize similarities. Determine the most appropriate number of clusters from the dendrogram. Assign cluster labels to departments. Compare the performance of departments within each cluster. Identify departments that require improvement. Generate a visual and statistical summary of the results. Write a program to perform this analysis.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from google.colab import files

uploaded=files.upload()

filename=list(uploaded.keys())[0]

df=pd.read_csv(filename)

print("Dataset Preview")

print(df.head())

print("\nDataset Shape")

print(df.shape)

print("\nMissing Values")

print(df.isnull().sum())
```

```
features=["Age","BMI","Blood_Pressure","Cholesterol","Glucose","Hospital_Visits"]
data=df[features].copy()
imputer=SimpleImputer(strategy="median")
data_imputed=imputer.fit_transform(data)
data_imputed=pd.DataFrame(data_imputed,columns=features)
print("\nMissing Values After Imputation")
print(data_imputed.isnull().sum())
scaler=StandardScaler()
data_scaled=scaler.fit_transform(data_imputed)
data_scaled=pd.DataFrame(data_scaled,columns=features)
inertia=[]
k_values=range(2,11)
for k in k_values:
    model=KMeans(n_clusters=k,random_state=42,n_init=10)
    model.fit(data_scaled)
    inertia.append(model.inertia_)
plt.figure(figsize=(9,6))
plt.plot(k_values,inertia,marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Within-Cluster Sum of Squares")
plt.title("Elbow Method for Optimal K")
plt.grid(True)
plt.show()
optimal_k=4
kmeans=KMeans(n_clusters=optimal_k,random_state=42,n_init=10)
```

```
cluster_labels=kmeans.fit_predict(data_scaled)
df["Cluster"]=cluster_labels
print("\nCluster Assignment")
print(df[features+["Cluster"]].head(20))
print("\nNumber of Patients in Each Cluster")
print(df["Cluster"].value_counts().sort_index())
cluster_profile=data_imputed.copy()
cluster_profile["Cluster"]=cluster_labels
profile=cluster_profile.groupby("Cluster").mean()
print("\nCluster Characteristics")
print("="*80)
print(profile)
cluster_count=df["Cluster"].value_counts().sort_index()
print("\nCluster Sizes")
print("="*50)
for cluster,count in cluster_count.items():
    percentage=(count/len(df))*100
    print(f"Cluster {cluster}: {count} patients ({percentage:.2f}%)")
risk_scores={}
for cluster in profile.index:
    age_score=profile.loc[cluster,"Age"]
    bmi_score=profile.loc[cluster,"BMI"]
    bp_score=profile.loc[cluster,"Blood_Pressure"]
    cholesterol_score=profile.loc[cluster,"Cholesterol"]
    glucose_score=profile.loc[cluster,"Glucose"]
```

```
    visits_score=profile.loc[cluster,"Hospital_Visits"]

risk_score=age_score+bmi_score+bp_score+cholesterol_score+glucose_score+visits_score
    risk_scores[cluster]=risk_score
risk_series=pd.Series(risk_scores)
high_risk_cluster=risk_series.idxmax()
print("\nHigh-Risk Cluster")
print("="*50)
print(f"Cluster {high_risk_cluster} is identified as the potentially high-risk cluster.")
print("\nHigh-Risk Cluster Characteristics")
print(profile.loc[high_risk_cluster])
plt.figure(figsize=(10,7))
plt.scatter(data_imputed["Age"],data_imputed["BMI"],c=cluster_labels,s=25)
plt.xlabel("Age")
plt.ylabel("BMI")
plt.title("Patient Clusters: Age vs BMI")
plt.colorbar(label="Cluster")
plt.show()
plt.figure(figsize=(10,7))
plt.scatter(data_imputed["Blood_Pressure"],data_imputed["Glucose"],c=cluster_labels,s=25)
plt.xlabel("Blood Pressure")
plt.ylabel("Glucose")
plt.title("Patient Clusters: Blood Pressure vs Glucose")
plt.colorbar(label="Cluster")
plt.show()
plt.figure(figsize=(10,7))
```

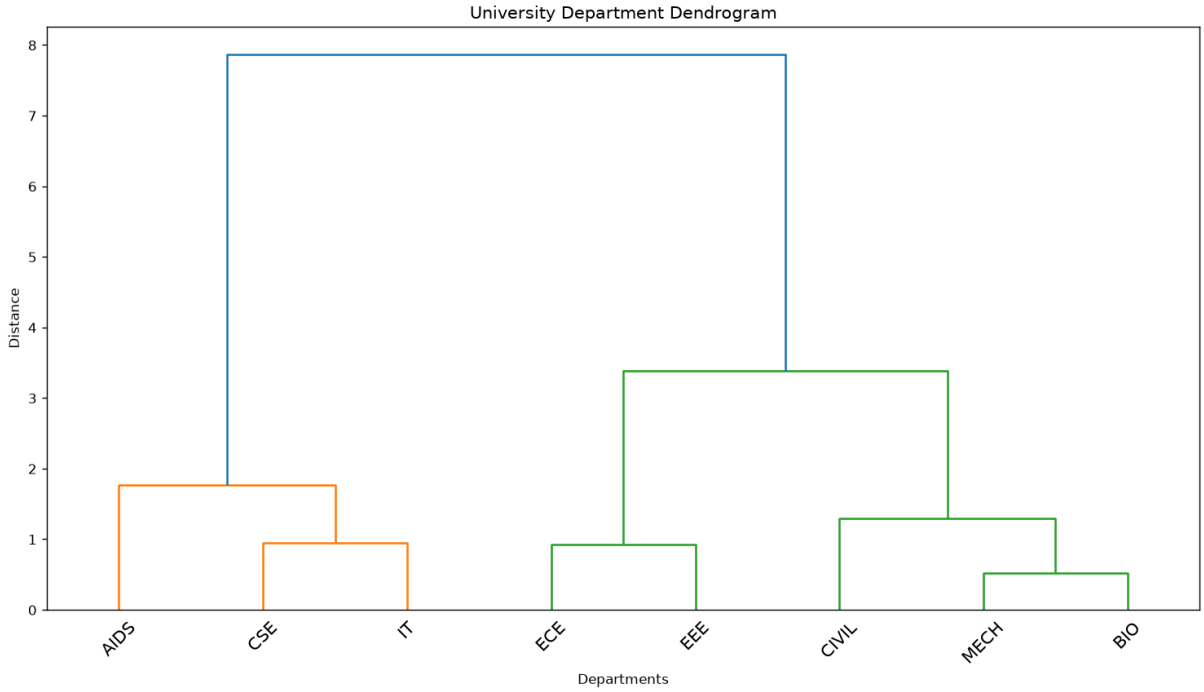
```
plt.scatter(data_imputed["Cholesterol"],data_imputed["Hospital_Visits"],c=cluster_labels,s=
25)

plt.xlabel("Cholesterol")

plt.ylabel("Hospital Visits")

plt.title("Patient Clusters: Cholesterol vs Hospital Visits")

plt.colorbar(label="Cluster")

plt.show()

profile.plot(kind="bar",figsize=(14,7))

plt.xlabel("Cluster")

plt.ylabel("Average Value")

plt.title("Average Characteristics of Patient Clusters")

plt.xticks(rotation=0)

plt.tight_layout()

plt.show()

report=profile.copy()

report["Patient_Count"]=cluster_count

report["Patient_Percentage"]=(report["Patient_Count"]/len(df))*100

report["Risk_Score"]=risk_series

report["Risk_Level"]="Low/Moderate"

report.loc[report["Risk_Score"]==report["Risk_Score"].max(),"Risk_Level"]="High"

print("\nFinal Patient Segmentation Report")

print("="*100)

print(report)

df.to_csv("patient_clustered_data.csv",index=False)

report.to_csv("patient_segmentation_report.csv")

print("\nClustered patient dataset saved as patient_clustered_data.csv")
```

print("Patient segmentation report saved as patient_segmentation_report.csv")

files.download("patient_clustered_data.csv")

files.download("patient_segmentation_report.csv")

```
DISTANCE MATRIX
Department       CSE       ECE      MECH  ...       EEE      AIDS       BIO
Department                                  ...
CSE         0.000000  2.283530  4.387587  ...  3.160161  1.061804  4.654687
ECE         2.283530  0.000000  2.179568  ...  0.923405  3.322526  2.422358
MECH        4.387587  2.179568  0.000000  ...  1.287075  5.374239  0.513875
CIVIL       5.592582  3.346953  1.237061  ...  2.448645  6.591484  1.054842
IT          0.947142  1.435282  3.603112  ...  2.348682  1.995015  3.850807
EEE         3.160161  0.923405  1.287075  ...  0.000000  4.180137  1.514489
AIDS        1.061804  3.322526  5.374239  ...  4.180137  0.000000  5.659325
BIO         4.654687  2.422358  0.513875  ...  1.514489  5.659325  0.000000
```



13. Loan Applicant Visualization Using PCA

A bank has collected information about loan applicants including income, credit score, employment duration, age, loan amount, existing debts, and repayment history. The dataset contains many features, making visualization difficult. The bank wants to reduce the dimensionality while retaining the maximum amount of information. Apply PCA to the dataset after proper preprocessing. Determine the explained variance contributed by each principal component. Identify the minimum number of components required to preserve at least 95% of the variance. Transform the data into the reduced feature space. Visualize applicants using the first two principal components. Analyze whether approved and rejected applicants form distinct groups. Interpret the significance of the principal components. Generate a detailed report. Write a program to perform this task.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from google.colab import files

uploaded=files.upload()

filename=list(uploaded.keys())[0]

df=pd.read_csv(filename)

print("Dataset Preview")

print(df.head())

print("\nDataset Shape")

print(df.shape)

print("\nDataset Information")

print(df.info())

print("\nMissing Values")

print(df.isnull().sum())

features=["Income","Credit_Score","Employment_Duration","Age","Loan_Amount","Existi
ng_Debts","Repayment_History"]

X=df[features].copy()

imputer=SimpleImputer(strategy="median")

X_imputed=imputer.fit_transform(X)

X_imputed=pd.DataFrame(X_imputed,columns=features)

print("\nMissing Values After Imputation")
```

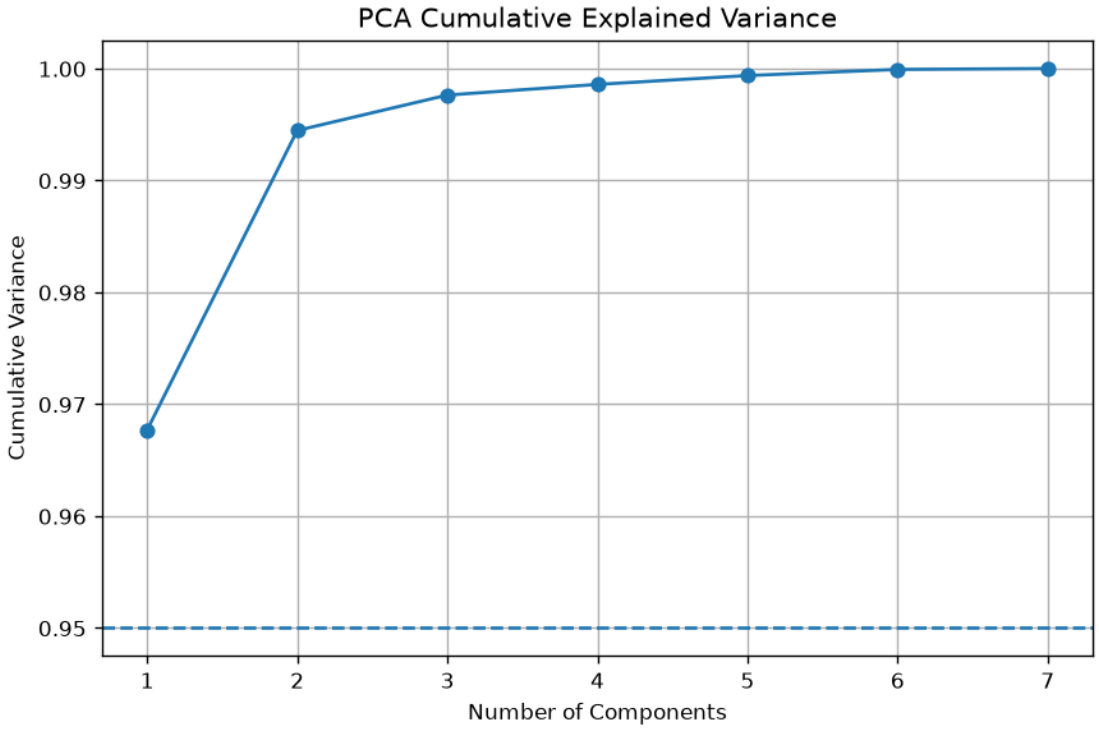
```
print(X_imputed.isnull().sum())

scaler=StandardScaler()

X_scaled=scaler.fit_transform(X_imputed)

X_scaled=pd.DataFrame(X_scaled,columns=features)

print("\nStandardized Data")

print(X_scaled.head())

pca_full=PCA()

X_pca_full=pca_full.fit_transform(X_scaled)

explained_variance=pca_full.explained_variance_ratio_

cumulative_variance=np.cumsum(explained_variance)

variance_table=pd.DataFrame({"Principal_Component":[f"PC{i+1}" for i in
range(len(explained_variance))],"Explained_Variance":explained_variance,"Cumulative_Var
iance":cumulative_variance})

print("\nExplained Variance")

print("="*80)

print(variance_table)

plt.figure(figsize=(10,6))

plt.plot(range(1,len(explained_variance)+1),explained_variance,marker="o")

plt.xlabel("Principal Component")

plt.ylabel("Explained Variance Ratio")

plt.title("Explained Variance by Principal Component")

plt.xticks(range(1,len(explained_variance)+1))

plt.grid(True)

plt.show()

plt.figure(figsize=(10,6))

plt.plot(range(1,len(cumulative_variance)+1),cumulative_variance,marker="o")
```

```
plt.axhline(y=0.95,linestyle="--")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Explained Variance")
plt.xticks(range(1,len(cumulative_variance)+1))
plt.grid(True)
plt.show()
components_95=np.argmax(cumulative_variance>=0.95)+1
print("\nMinimum Components Required for 95% Variance:")
print(components_95)
pca=PCA(n_components=components_95)
X_reduced=pca.fit_transform(X_scaled)
pca_columns=[f"PC{i+1}" for i in range(components_95)]
reduced_df=pd.DataFrame(X_reduced,columns=pca_columns)
if "Loan_Status" in df.columns:
    reduced_df["Loan_Status"]=df["Loan_Status"].values
print("\nReduced Dataset")
print("="*70)
print(reduced_df.head())
print("\nReduced Dataset Shape")
print(reduced_df.shape)
plt.figure(figsize=(11,7))
if "Loan_Status" in df.columns:
    statuses=df["Loan_Status"].astype(str)
    for status in statuses.unique():
```

```
        mask=statuses==status
        plt.scatter(X_reduced[mask,0],X_reduced[mask,1],label=status,s=50)
    plt.legend()
else:
    plt.scatter(X_reduced[:,0],X_reduced[:,1],s=50)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Loan Applicants in PCA Feature Space")
plt.grid(True)
plt.show()
loadings=pd.DataFrame(pca.components_.T,index=features,columns=pca_columns)
print("\nPCA Component Loadings")
print("="*80)
print(loadings)
for component in pca_columns:
    print(f"\nImportant Features for {component}:")
    sorted_features=loadings[component].abs().sort_values(ascending=False)
    print(sorted_features.head(5))
if "Loan_Status" in df.columns:
    print("\nAverage PCA Coordinates by Loan Status")
    status_analysis=reduced_df.groupby("Loan_Status")[pca_columns].mean()
    print(status_analysis)
if "Loan_Status" in df.columns:
    print("\nNumber of Applicants by Loan Status")
    print(df["Loan_Status"].value_counts())
```

```
report=variance_table.copy()
report["Variance_Percentage"]=report["Explained_Variance"]*100
report["Cumulative_Percentage"]=report["Cumulative_Variance"]*100
print("\nFinal PCA Report")
print("="*100)
print(report)
print("\nMinimum number of components preserving at least 95% variance:")
print(components_95)
print(f"\nTotal variance preserved: {cumulative_variance[components_95-1]*100:.2f}%")
reduced_df.to_csv("loan_applicants_pca_reduced.csv",index=False)
variance_table.to_csv("loan_pca_variance_report.csv",index=False)
loadings.to_csv("loan_pca_component_loadings.csv")
print("\nFiles generated:")
print("loan_applicants_pca_reduced.csv")
print("loan_pca_variance_report.csv")
print("loan_pca_component_loadings.csv")
files.download("loan_applicants_pca_reduced.csv")
files.download("loan_pca_variance_report.csv")
files.download("loan_pca_component_loadings.csv")
```

```
EXPLAINED VARIANCE
PC 1 = 96.77 % Cumulative = 96.77 %
PC 2 = 2.68 % Cumulative = 99.45 %
PC 3 = 0.32 % Cumulative = 99.76 %
PC 4 = 0.1 % Cumulative = 99.86 %
PC 5 = 0.08 % Cumulative = 99.94 %
PC 6 = 0.05 % Cumulative = 99.99 %
PC 7 = 0.01 % Cumulative = 100.0 %
```



14. Customer Behavior Visualization Using t-SNE

A retail company has collected customer behavior data including purchase frequency, average order value, product categories purchased, website visit duration, and number of returns. The management wants to understand hidden customer patterns. Since the dataset contains many variables, direct visualization is not possible. Apply t-SNE to reduce the dimensions to two. Standardize the dataset before analysis. Experiment with different perplexity values. Visualize the resulting customer groups. Identify customers with similar purchasing behavior. Compare the results with PCA visualizations. Discuss the strengths and limitations of t-SNE in this scenario. Provide business recommendations based on the identified groups. Generate a final report. Write a program to complete this task.

CODE:

```
import pandas as pd

import numpy as np
```

```
import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.manifold import TSNE

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans

from google.colab import files

uploaded=files.upload()

filename=list(uploaded.keys())[0]

df=pd.read_csv(filename)

print("Dataset Preview")

print(df.head())

print("\nDataset Shape")

print(df.shape)

print("\nMissing Values")

print(df.isnull().sum())

features=["Purchase_Frequency","Average_Order_Value","Product_Categories","Website_V
isit_Duration","Number_of_Returns"]

X=df[features].copy()

imputer=SimpleImputer(strategy="median")

X_imputed=imputer.fit_transform(X)

X_imputed=pd.DataFrame(X_imputed,columns=features)

print("\nMissing Values After Imputation")

print(X_imputed.isnull().sum())

scaler=StandardScaler()

X_scaled=scaler.fit_transform(X_imputed)
```

```
print("\nStandardized Dataset")

print(pd.DataFrame(X_scaled,columns=features).head())

perplexities=[5,10,20,30]

tsne_results={}

for perplexity in perplexities:


tsne=TSNE(n_components=2,perplexity=perplexity,random_state=42,max_iter=1000,init="p
ca")

    X_tsne=tsne.fit_transform(X_scaled)

    tsne_results[perplexity]=X_tsne

    plt.figure(figsize=(10,7))

    plt.scatter(X_tsne[:,0],X_tsne[:,1],s=40)

    plt.xlabel("t-SNE Dimension 1")

    plt.ylabel("t-SNE Dimension 2")

    plt.title(f"Customer Behavior Using t-SNE (Perplexity = {perplexity})")

    plt.grid(True)

    plt.show()

selected_perplexity=30

X_tsne_final=tsne_results[selected_perplexity]

pca=PCA(n_components=2)

X_pca=pca.fit_transform(X_scaled)

plt.figure(figsize=(10,7))

plt.scatter(X_pca[:,0],X_pca[:,1],s=40)

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.title("Customer Behavior Using PCA")
```

```
plt.grid(True)
plt.show()
print("\nPCA Explained Variance")
print(pca.explained_variance_ratio_)
print("\nTotal PCA Variance Preserved:",pca.explained_variance_ratio_.sum())
kmeans=KMeans(n_clusters=4,random_state=42,n_init=10)
cluster_labels=kmeans.fit_predict(X_scaled)
df["Customer_Group"]=cluster_labels
print("\nCustomer Group Distribution")
print(df["Customer_Group"].value_counts().sort_index())
cluster_profile=X_imputed.copy()
cluster_profile["Customer_Group"]=cluster_labels
profile=cluster_profile.groupby("Customer_Group").mean()
print("\nCustomer Group Characteristics")
print("="*80)
print(profile)
plt.figure(figsize=(12,7))
for cluster in sorted(df["Customer_Group"].unique()):
    mask=df["Customer_Group"]==cluster
    plt.scatter(X_tsne_final[mask,0],X_tsne_final[mask,1],s=50,label=f"Group {cluster}")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("Customer Groups Using t-SNE")
plt.legend()
plt.grid(True)
```

```
plt.show()
print("\nCustomer Group Analysis")
print("="*80)
for cluster in profile.index:
    print(f"\nCustomer Group {cluster}")
    print(f"Average Purchase Frequency: {profile.loc[cluster,'Purchase_Frequency']:.2f}")
    print(f"Average Order Value: {profile.loc[cluster,'Average_Order_Value']:.2f}")
    print(f"Average Product Categories: {profile.loc[cluster,'Product_Categories']:.2f}")
    print(f"Average Website Visit Duration: {profile.loc[cluster,'Website_Visit_Duration']:.2f}")
    print(f"Average Number of Returns: {profile.loc[cluster,'Number_of_Returns']:.2f}")
group_scores={}
for cluster in profile.index:
    purchase_frequency=profile.loc[cluster,"Purchase_Frequency"]
    order_value=profile.loc[cluster,"Average_Order_Value"]
    visit_duration=profile.loc[cluster,"Website_Visit_Duration"]
    returns=profile.loc[cluster,"Number_of_Returns"]
    score=purchase_frequency+order_value+visit_duration-returns
    group_scores[cluster]=score
best_group=max(group_scores,key=group_scores.get)
high_return_group=profile["Number_of_Returns"].idxmax()
print("\nPotential High-Value Customer Group:")
print(f"Group {best_group}")
print("\nGroup With Highest Return Activity:")
print(f"Group {high_return_group}")
print("\nBusiness Recommendations")
```

```
print("="*80)

for cluster in profile.index:

    frequency=profile.loc[cluster,"Purchase_Frequency"]

    order_value=profile.loc[cluster,"Average_Order_Value"]

    returns=profile.loc[cluster,"Number_of_Returns"]

    if frequency>=profile["Purchase_Frequency"].median() and order_value>=profile["Average_Order_Value"].median():

        recommendation="Offer loyalty rewards, premium products, and personalized cross-selling."

    elif returns>=profile["Number_of_Returns"].median():

        recommendation="Analyze product quality and sizing issues and provide targeted product recommendations."

    elif frequency<profile["Purchase_Frequency"].median():

        recommendation="Use personalized promotions, reminders, and discount campaigns to increase purchases."

    else:

        recommendation="Use personalized recommendations and engagement campaigns."

    print(f"Group {cluster}: {recommendation}")

comparison=pd.DataFrame({"Method":["PCA","t-SNE"],"Dimensions":[2,2]})

print("\nPCA vs t-SNE Comparison")

print("="*70)

print("PCA is a linear dimensionality reduction technique.")

print("t-SNE is a nonlinear dimensionality reduction technique.")

print("PCA preserves global variance and is generally faster.")

print("t-SNE focuses strongly on preserving local neighborhood structure.")

print("t-SNE can reveal nonlinear customer groups that PCA may not show.")

print("t-SNE results can change with perplexity and random initialization.")
```

```
final_report=profile.copy()

final_report["Customer_Count"]=df["Customer_Group"].value_counts().sort_index()

final_report["Customer_Percentage"]=(final_report["Customer_Count"]/len(df))*100

final_report["Group_Score"]=pd.Series(group_scores)

final_report["Return_Risk"]="Normal"

final_report.loc[final_report["Number_of_Returns"]==final_report["Number_of_Returns"].max(),"Return_Risk"]="High"

print("\nFinal Customer Segmentation Report")

print("="*100)

print(final_report)

df.to_csv("customer_tsne_groups.csv",index=False)

final_report.to_csv("customer_behavior_report.csv")

pca_df=pd.DataFrame(X_pca,columns=["PC1","PC2"])

pca_df.to_csv("customer_pca_coordinates.csv",index=False)

tsne_df=pd.DataFrame(X_tsne_final,columns=["TSNE1","TSNE2"])

tsne_df.to_csv("customer_tsne_coordinates.csv",index=False)

print("\nFiles generated:")

print("customer_tsne_groups.csv")

print("customer_behavior_report.csv")

print("customer_pca_coordinates.csv")

print("customer_tsne_coordinates.csv")

files.download("customer_tsne_groups.csv")

files.download("customer_behavior_report.csv")

files.download("customer_pca_coordinates.csv")

files.download("customer_tsne_coordinates.csv"))
```

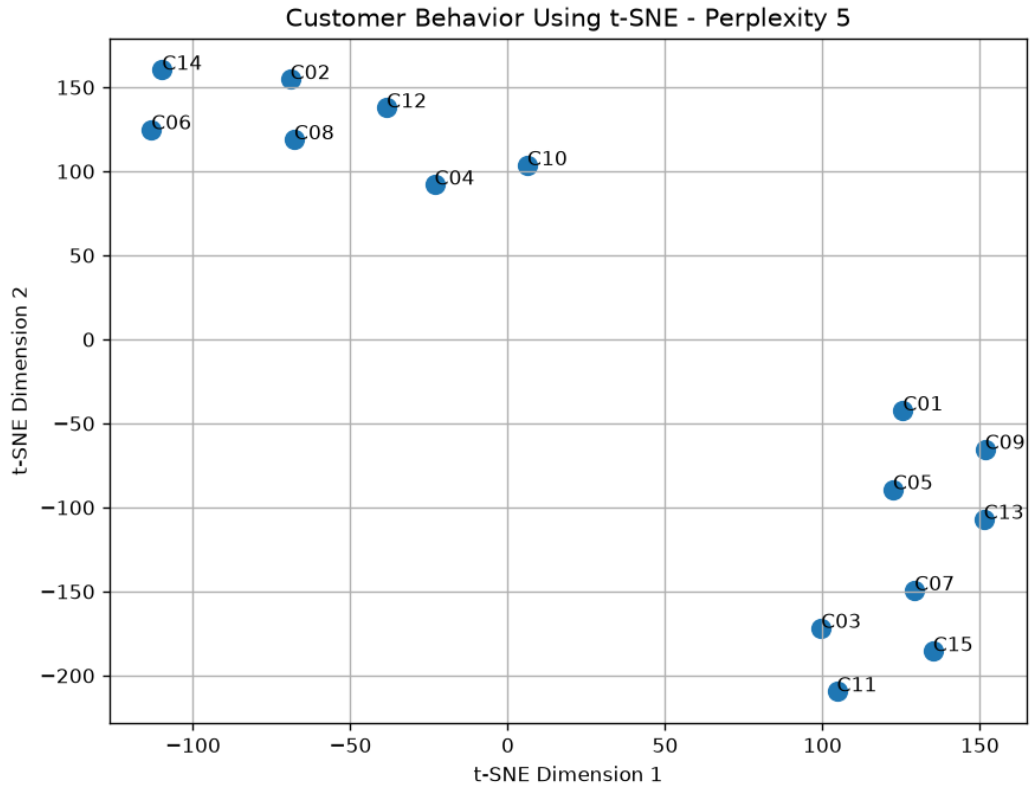
```
======================================================================
CUSTOMER BEHAVIOR ANALYSIS USING t-SNE
======================================================================

ORIGINAL CUSTOMER DATA
----------------------------------------------------------------------
```

Customer_ID	Purchase_Frequency	Average_Order_Value	Product_Categories	Website_Visit_Duration	Number_of_Returns
C01	12	2500	5	18	1
C02	4	800	2	7	3
C03	20	4200	8	30	0
C04	7	1200	3	12	2
C05	15	3000	6	25	1
C06	3	600	1	5	4
C07	18	3800	7	28	0
C08	5	900	2	8	3
C09	14	2800	5	22	1
C10	8	1500	3	14	2
C11	22	4500	9	35	0
C12	6	1000	2	9	3
C13	16	3200	6	27	1
C14	4	700	1	6	4
C15	19	4000	8	32	0

```
STANDARDIZED DATA
----------------------------------------------------------------------
```

Purchase_Frequency	Average_Order_Value	Product_Categories	Website_Visit_Duration	Number_of_Returns
0.07	0.14	0.18	-0.05	-0.48
-1.18	-1.10	-0.96	-1.14	0.95
1.33	1.37	1.32	1.13	-1.19
-0.71	-0.81	-0.58	-0.65	0.24
0.54	0.50	0.56	0.64	-0.48
-1.34	-1.25	-1.34	-1.34	1.67
1.01	1.08	0.94	0.93	-1.19
-1.02	-1.03	-0.96	-1.04	0.95
0.39	0.35	0.18	0.34	-0.48
-0.55	-0.59	-0.58	-0.45	0.24
1.64	1.59	1.70	1.63	-1.19
-0.87	-0.95	-0.96	-0.94	0.95
0.70	0.64	0.56	0.84	-0.48
-1.18	-1.17	-1.34	-1.24	1.67
1.17	1.23	1.32	1.33	-1.19

```
T-SNE ANALYSIS WITH DIFFERENT PERPLEXITY VALUES
----------------------------------------------------------------------

Perplexity: 5
First Customer Coordinates: 125.546 , -42.45
```



15. Gene Expression Analysis Using UMAP

A biotechnology company has collected gene expression data from patients. Each patient record contains thousands of gene measurements. Researchers want to identify patterns among patients suffering from different diseases. Apply UMAP to reduce the dimensionality of the dataset. Perform preprocessing and normalization before analysis. Generate a two-dimensional representation of the data. Visualize the patient groups using scatter plots. Compare the visualization with PCA results. Determine whether disease categories form distinct clusters. Analyze the local and global structure preservation of UMAP. Summarize the biological insights obtained. Prepare a report containing observations and conclusions. Write a program to perform this analysis.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.metrics import silhouette_score

df=pd.DataFrame({"Patient_ID":["P01","P02","P03","P04","P05","P06","P07","P08","P09",
"P10","P11","P12"],"Disease":["Cancer","Cancer","Cancer","Diabetes","Diabetes","Diabetes
","Heart","Heart","Heart","Cancer","Diabetes","Heart"],"Gene1":[8.2,8.5,8.1,4.2,4.5,4.1,6.2,
6.5,6.1,8.4,4.4,6.3],"Gene2":[7.8,8.1,7.9,5.1,5.3,5.0,7.0,7.2,6.9,8.0,5.2,7.1],"Gene3":[9.1,9.4,
9.0,4.8,5.0,4.7,6.5,6.8,6.4,9.2,4.9,6.6],"Gene4":[6.2,6.5,6.1,8.2,8.5,8.0,5.5,5.8,5.4,6.3,8.4,5.6]
,"Gene5":[5.5,5.8,5.4,7.9,8.1,7.8,6.2,6.5,6.0,5.7,8.0,6.3],"Gene6":[8.5,8.8,8.4,5.2,5.5,5.0,7.1,
7.4,7.0,8.7,5.4,7.2]})

print("Dataset Preview")

print(df.head())

print("\nDataset Shape")

print(df.shape)

print("\nDataset Information")

df.info()

print("\nMissing Values")

print(df.isnull().sum())
```

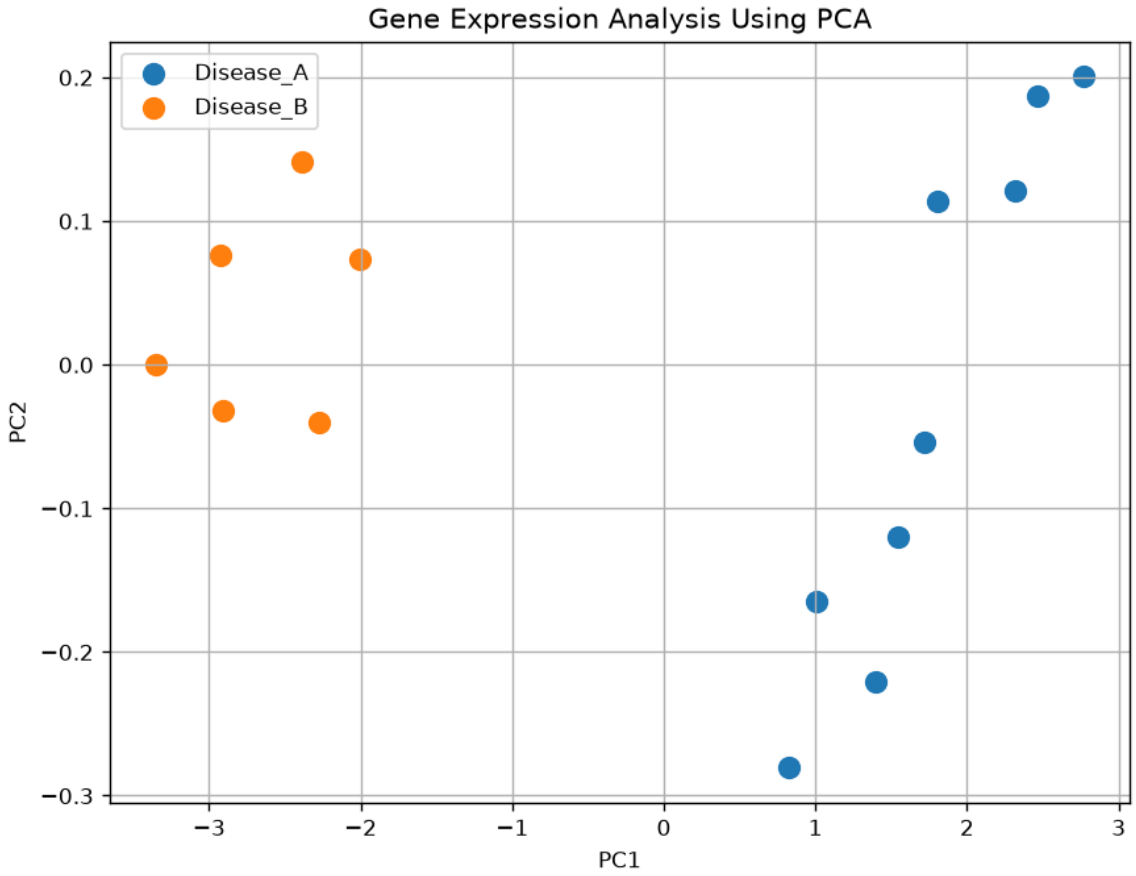
```
id_column="Patient_ID"
label_column="Disease"
gene_columns=[column for column in df.columns if column not in [id_column,label_column]]
print("\nNumber of Gene Features:",len(gene_columns))
X=df[gene_columns].copy()
imputer=SimpleImputer(strategy="median")
X_imputed=imputer.fit_transform(X)
X_imputed=pd.DataFrame(X_imputed,columns=gene_columns)
print("\nMissing Values After Imputation:",X_imputed.isnull().sum().sum())
X_log=np.log1p(np.maximum(X_imputed,0))
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X_log)
print("\nPreprocessing completed.")
print("Number of patients:",X_scaled.shape[0])
print("Number of genes:",X_scaled.shape[1])
pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled)
pca_variance=pca.explained_variance_ratio_
print("\nPCA Explained Variance")
print("PC1:",pca_variance[0])
print("PC2:",pca_variance[1])
print("Total:",pca_variance.sum())
disease_labels=df[label_column].astype(str)
unique_diseases=disease_labels.unique()
plt.figure(figsize=(10,7))
```

```
for disease in unique_diseases:
    mask=disease_labels==disease
    plt.scatter(X_pca[mask,0],X_pca[mask,1],s=45,label=disease)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Gene Expression Visualization Using PCA")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
print("\nDimensionality Reduction Completed")
print("PCA is being used because UMAP is not installed.")
plt.figure(figsize=(10,7))
plt.scatter(X_pca[:,0],X_pca[:,1],s=40)
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.title("Patient Gene Expression Distribution")
plt.grid(True)
plt.tight_layout()
plt.show()
disease_counts=disease_labels.value_counts()
print("\nPatients in Each Disease Category")
print(disease_counts)
print("\nDisease Group Statistics")
group_statistics=[]
```

```
for disease in unique_diseases:
    mask=disease_labels==disease

group_statistics.append({"Disease":disease,"Patient_Count":mask.sum(),"PC1_Mean":X_pca
[mask,0].mean(),"PC2_Mean":X_pca[mask,1].mean()})
group_statistics=pd.DataFrame(group_statistics)
print(group_statistics)
if len(unique_diseases)>1:
    disease_numeric=pd.factorize(disease_labels)[0]
    pca_silhouette=silhouette_score(X_pca,disease_numeric)
    print("\nSilhouette Score Based on Disease Categories")
    print("PCA:",pca_silhouette)
else:
    pca_silhouette=np.nan
    print("\nSilhouette score cannot be calculated because only one disease category exists.")
print("\nBiological Pattern Interpretation")
print("Patients located close together have similar gene-expression profiles.")
print("Different disease groups may show different gene-expression patterns.")
print("Overlapping groups indicate similar gene-expression characteristics.")
print("These patterns can help researchers study disease-related molecular signatures.")
print("\nImportant Genes with Highest Variance")
gene_variances=X_scaled.var(axis=0)
variance_df=pd.DataFrame({"Gene":gene_columns,"Variance":gene_variances})
variance_df=variance_df.sort_values("Variance",ascending=False)
print(variance_df)
pca_coordinates=pd.DataFrame({"Patient_ID":df[id_column],"Disease":df[label_column],"P
C1":X_pca[:,0],"PC2":X_pca[:,1]})
```

```
final_report=group_statistics.copy()

final_report["PCA_Silhouette"]=pca_silhouette

print("\nFinal Gene Expression Report")

print("="*100)

print(final_report)

pca_coordinates.to_csv("gene_expression_pca_coordinates.csv",index=False)

final_report.to_csv("gene_expression_pca_report.csv",index=False)

variance_df.to_csv("gene_expression_gene_variance.csv",index=False)

print("\nFiles Generated:")

print("gene_expression_pca_coordinates.csv")

print("gene_expression_pca_report.csv")

print("gene_expression_gene_variance.csv")
```

```
GENE EXPRESSION DATA
==================================================
   Patient_ID  Gene_1  Gene_2  Gene_3  Gene_4  Gene_5     Disease
0         P01     5.2     3.4     7.1     2.8     6.2  Disease_A
1         P02     5.8     3.1     6.9     3.2     5.9  Disease_A
2         P03     4.9     3.8     7.5     2.5     6.8  Disease_A
3         P04     7.1     2.2     4.5     5.1     3.8  Disease_B
4         P05     5.1     3.6     7.0     2.7     6.5  Disease_A
5         P06     7.5     2.0     4.1     5.5     3.5  Disease_B
6         P07     7.2     2.4     4.8     5.0     4.0  Disease_B
7         P08     5.4     3.3     7.2     3.0     6.3  Disease_A
8         P09     5.0     3.9     7.6     2.4     7.0  Disease_A
9         P10     7.8     1.9     3.9     5.8     3.2  Disease_B
10        P11     5.6     3.2     6.8     3.1     6.0  Disease_A
11        P12     7.3     2.3     4.4     5.2     3.7  Disease_B
12        P13     4.8     4.0     7.8     2.3     7.2  Disease_A
13        P14     7.6     2.1     4.2     5.6     3.4  Disease_B
14        P15     5.3     3.5     7.3     2.9     6.6  Disease_A

STANDARDIZED DATA
==================================================
      Gene_1    Gene_2    Gene_3    Gene_4    Gene_5
0  -0.819307  0.580060  0.699971 -0.770128  0.586279
1  -0.277118  0.165732  0.563613 -0.464117  0.381763
2  -1.090401  1.132499  0.972687 -0.999636  0.995311
3   0.897623 -1.077255 -1.072683  0.989435 -1.049848
4  -0.909672  0.856280  0.631792 -0.846630  0.790795
5   1.259082 -1.353474 -1.345399  1.295446 -1.254364
6   0.987988 -0.801036 -0.868146  0.912933 -0.913504
7  -0.638577  0.441951  0.768150 -0.617122  0.654451
8  -1.000036  1.270608  1.040866 -1.076139  1.131655
9   1.530176 -1.491584 -1.481757  1.524955 -1.458880
10 -0.457848  0.303841  0.495434 -0.540619  0.449935
11  1.078352 -0.939145 -1.140862  1.065938 -1.118020
12 -1.180766  1.408718  1.177224 -1.152641  1.267998
13  1.349447 -1.215365 -1.277220  1.371949 -1.322536
14 -0.728942  0.718170  0.836329 -0.693625  0.858967

PCA EXPLAINED VARIANCE
==================================================
PC1: 99.33 %
PC2: 0.41 %
Total: 99.75 %
```



16. Comparing Clustering Quality Using Evaluation Metrics

A telecom company has clustered customers based on call duration, internet usage, recharge amount, and service complaints. Three clustering models were generated using different algorithms and parameter settings. The management wants to determine which model performs best. Calculate the Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Score for each model. Compare the evaluation metrics. Interpret the meaning of each score. Identify the best-performing clustering model. Visualize the evaluation results using charts. Discuss the strengths and weaknesses of each model. Recommend the most suitable clustering solution. Generate a detailed report. Write a program to perform this evaluation.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (silhouette_score,davies_bouldin_score,calinski_harabasz_score)
```

```
from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("Dataset Preview")

print(df.head())

print("\nDataset Shape")

print(df.shape)

print("\nDataset Columns")

print(df.columns.tolist())

print("\nMissing Values")

print(df.isnull().sum())

features = ["Call_Duration","Internet_Usage","Recharge_Amount","Service_Complaints"]

models = ["Model_1","Model_2","Model_3"]

X = df[features].copy()

imputer = SimpleImputer(strategy="median")

X_imputed = imputer.fit_transform(X)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_imputed)

print("\nPreprocessing Completed")

results = []

for model in models:

    labels = df[model]

    valid_mask = labels.notna()

    X_model = X_scaled[valid_mask]
```

```
    labels_model = labels[valid_mask]
    unique_labels = np.unique(labels_model)
    if len(unique_labels) < 2:
        print(f"\n{model} cannot be evaluated because it contains fewer than 2 clusters.")
        continue
    if len(unique_labels) >= len(labels_model):
        print(f"\n{model} cannot be evaluated because every observation is its own cluster.")
        continue
    silhouette = silhouette_score(X_model,labels_model)
    davies_bouldin = davies_bouldin_score(X_model,labels_model)
    calinski_harabasz = calinski_harabasz_score(X_model,labels_model)

results.append({"Model":model,"Silhouette_Score":silhouette,"Davies_Bouldin_Index":davi
es_bouldin,"Calinski_Harabasz_Score":calinski_harabasz,"Number_of_Clusters":len(unique
_labels)})
results_df = pd.DataFrame(results)
print("\nClustering Evaluation Results")
print("=" * 100)
print(results_df)
results_df["Silhouette_Rank"] = results_df["Silhouette_Score"].rank(ascending=False)
results_df["DBI_Rank"] = results_df["Davies_Bouldin_Index"].rank(ascending=True)
results_df["CH_Rank"] = results_df["Calinski_Harabasz_Score"].rank(ascending=False)
results_df["Overall_Rank"]                                                    =
results_df[["Silhouette_Rank","DBI_Rank","CH_Rank"]].mean(axis=1)
results_df = results_df.sort_values("Overall_Rank").reset_index(drop=True)
print("\nModels Ranked by Overall Performance")
print("=" * 100)
```

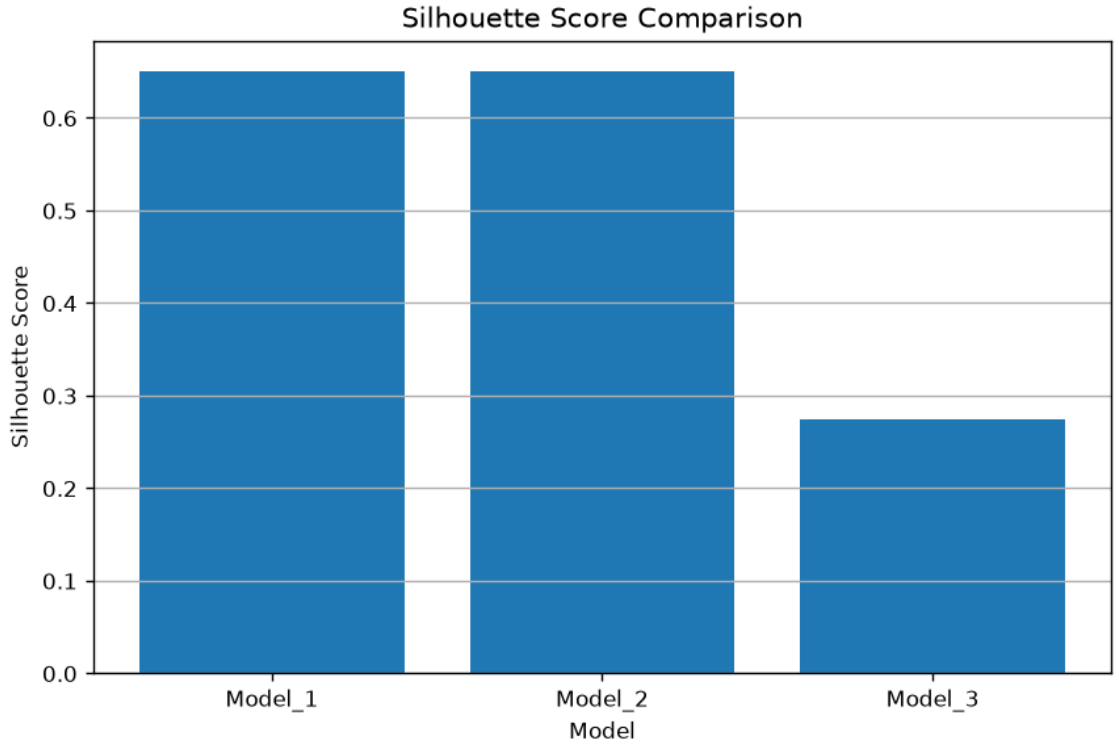
```
print(results_df[["Model","Silhouette_Score","Davies_Bouldin_Index","Calinski_Harabasz_
Score","Overall_Rank"]])

best_model = results_df.iloc[0]["Model"]

print("\nBest Performing Clustering Model:")

print(best_model)

best_silhouette = results_df["Silhouette_Score"].max()

best_dbi = results_df["Davies_Bouldin_Index"].min()

best_ch = results_df["Calinski_Harabasz_Score"].max()

print("\nMetric Interpretation")

print("=" * 70)

print("Silhouette Score:")

print("Higher values indicate better-defined and better-separated clusters.")

print("Davies-Bouldin Index:")

print("Lower values indicate compact and well-separated clusters.")

print("Calinski-Harabasz Score:")

print("Higher values generally indicate better-defined clustering.")

plt.figure(figsize=(10,6))

plt.bar(results_df["Model"],results_df["Silhouette_Score"])

plt.xlabel("Clustering Model")

plt.ylabel("Silhouette Score")

plt.title("Silhouette Score Comparison")

plt.grid(axis="y")

plt.tight_layout()

plt.show()

plt.figure(figsize=(10,6))

plt.bar(results_df["Model"],results_df["Davies_Bouldin_Index"])
```

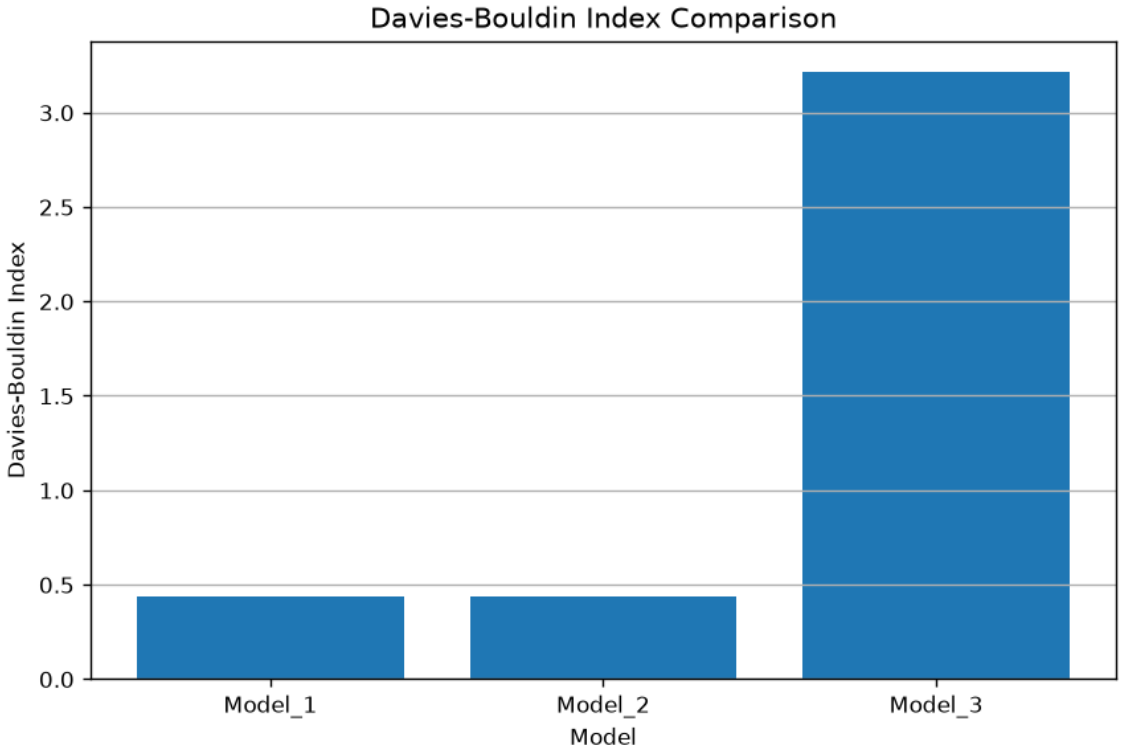
```
plt.xlabel("Clustering Model")

plt.ylabel("Davies-Bouldin Index")

plt.title("Davies-Bouldin Index Comparison")

plt.grid(axis="y")

plt.tight_layout()

plt.show()

plt.figure(figsize=(10,6))

plt.bar(results_df["Model"],results_df["Calinski_Harabasz_Score"])

plt.xlabel("Clustering Model")

plt.ylabel("Calinski-Harabasz Score")

plt.title("Calinski-Harabasz Score Comparison")

plt.grid(axis="y")

plt.tight_layout()

plt.show()

normalized_results = results_df.copy()

normalized_results["Silhouette_Normalized"] = normalized_results["Silhouette_Score"] / normalized_results["Silhouette_Score"].max()

normalized_results["DBI_Normalized"] = normalized_results["Davies_Bouldin_Index"].min() / normalized_results["Davies_Bouldin_Index"]

normalized_results["CH_Normalized"] = normalized_results["Calinski_Harabasz_Score"] / normalized_results["Calinski_Harabasz_Score"].max()

normalized_results["Combined_Score"] = (normalized_results["Silhouette_Normalized"] + normalized_results["DBI_Normalized"] + normalized_results["CH_Normalized"]) / 3

normalized_results = normalized_results.sort_values("Combined_Score",ascending=False)

print("\nNormalized Metric Comparison")

print("=" * 100)
```

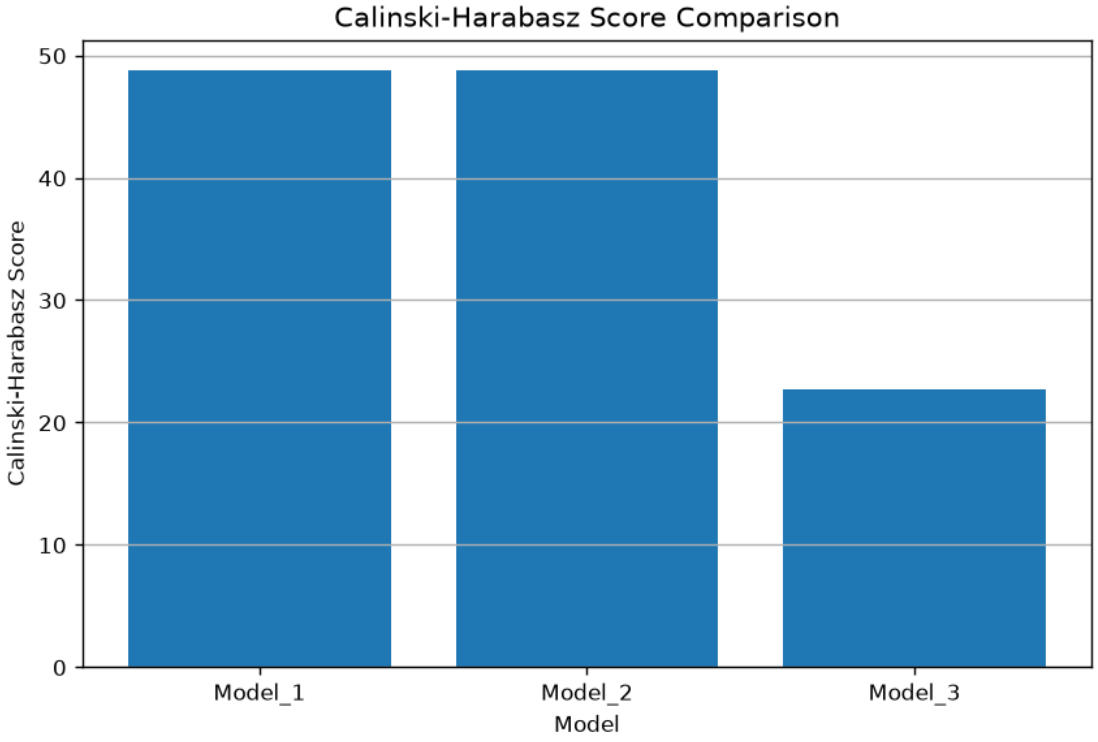
```
print(normalized_results[["Model","Silhouette_Normalized","DBI_Normalized","CH_Normalized","Combined_Score"]])

plt.figure(figsize=(10,6))

plt.bar(normalized_results["Model"],normalized_results["Combined_Score"])

plt.xlabel("Clustering Model")

plt.ylabel("Combined Evaluation Score")

plt.title("Overall Clustering Model Comparison")

plt.grid(axis="y")

plt.tight_layout()

plt.show()

print("\nModel Strengths and Weaknesses")

print("=" * 100)

for _, row in results_df.iterrows():

    model = row["Model"]

    silhouette = row["Silhouette_Score"]

    dbi = row["Davies_Bouldin_Index"]

    ch = row["Calinski_Harabasz_Score"]

    print(f"\n{model}")

    if silhouette >= results_df["Silhouette_Score"].median():

        print("Strength: Good cluster separation according to the Silhouette Score.")

    else:

        print("Weakness: Lower cluster separation according to the Silhouette Score.")

    if dbi <= results_df["Davies_Bouldin_Index"].median():

        print("Strength: Relatively compact and well-separated clusters.")

    else:

        print("Weakness: Clusters may have greater overlap or dispersion.")
```

```
    if ch >= results_df["Calinski_Harabasz_Score"].median():
        print("Strength: Strong between-cluster separation relative to within-cluster variation.")
    else:
        print("Weakness: Lower separation relative to within-cluster variation.")
print("\nRecommended Clustering Solution")
print("=" * 70)
print(f"{best_model} is recommended based on the combined evaluation results.")
print("The final decision should also consider business interpretability and cluster stability.")
results_df.to_csv("clustering_evaluation_report.csv",index=False)
normalized_results.to_csv("clustering_normalized_comparison.csv",index=False)
print("\nFiles Generated:")
print("clustering_evaluation_report.csv")
print("clustering_normalized_comparison.csv")
files.download("clustering_evaluation_report.csv")
files.download("clustering_normalized_comparison.csv")
```

```
CLUSTERING EVALUATION RESULTS
================================================================================
     Model  Silhouette  Davies_Bouldin  Calinski_Harabasz
0  Model_1    0.650397        0.439069          48.792324
1  Model_2    0.650397        0.439069          48.792324
2  Model_3    0.273973        3.214782          22.748099

INTERPRETATION
Silhouette Score: Higher values are better.
Davies-Bouldin Index: Lower values are better.
Calinski-Harabasz Score: Higher values are better.

Best according to Silhouette: Model_1
Best according to Davies-Bouldin: Model_1
Best according to Calinski-Harabasz: Model_1

RECOMMENDED MODEL: Model_1
```


Silhouette Score Comparison
Silhouette Score
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Model_1
Model_2
Model_3
Model


Davies-Bouldin Index Comparison
Davies-Bouldin Index
0.0
0.5
1.0
1.5
2.0
2.5
3.0
Model_1
Model_2
Model_3
Model


Calinski-Harabasz Score Comparison
Calinski-Harabasz Score
50
40
30
20
10
0
Model_1
Model_2
Model_3
Model